

Project Report: DL-GenAI Project

Student Name: Ashrita Khanta

Roll No: 22f1000828

Course Code: BSDA2001P

Course Name: Introduction to DL and GenAI Project

Date: July 28, 2026

1. Abstract / Summary

For this project I took part in the Kaggle competition called Smart MCQ Solver Challenge. In this task the model reads an academic multiple choice question and has to pick the correct answer out of five options (A to E). The score used is MAP@3, so I do not give just one answer. I give my top three guesses in order.

The main design choice I made was to treat this as an option scoring problem and not as a five class classification problem. Every question is turned into five small texts of the form "prompt [SEP] option". One shared head gives a score to each of the five texts. Then the five scores are put together into one vector of length 5 and trained with cross entropy loss. All three of my models use exactly this same setup, so comparing them is fair.

I built three models: a fine tuned DeBERTa v3 transformer, my own TextCNN, and a bidirectional LSTM written from scratch. I ran three experiments per model (nine runs in total) and logged all of them to Weights and Biases.

My result was not what I expected. Both models I wrote from scratch got a perfect validation MAP@3 of 1.0000, while the fine tuned DeBERTa only reached 0.9200. That is the opposite of the normal transfer learning story. But my final submission got only **0.75810 on the Kaggle leaderboard (rank 164)**, so the perfect validation score was misleading. The dataset is small (2,000 rows), very templated, has only 2,980 unique words with a 0% out of vocabulary rate, and has 242 duplicate prompts that leak across a random split. So a small model can reach 1.0 on validation just by memorising. My most useful single finding was that freezing the embedding layer of DeBERTa more than doubled its score, from 0.4039 to 0.9200.

2. Introduction

Problem Statement

The competition gives academic multiple choice questions from science, physics and philosophy. Each row has a question prompt, five options in columns A to E, and in the training set it also has the letter of the correct answer. I have to predict the answer for each test question.

The metric is MAP@3. The correct letter gets 1 point if I put it first, 1/2 if second, 1/3 if third, and 0 otherwise. So the model must give a full probability distribution and not a hard label, and the random baseline is $(1 + 1/2 + 1/3) / 5 = 0.3667$ and not 0.20. Anything below 0.3667 is worse than guessing, which is exactly what my first DeBERTa run turned out to be.

Project Objective

- To use the same option scoring setup for all three models, so that I am really comparing architectures and not different data pipelines.
- To write an LSTM cell from scratch, with all four gates and both directions, without using nn.LSTM.
- To run three logged configurations per model, and to choose max_len, vocab_size and learning rate using evidence from my EDA instead of just copying default values.
- To measure simple baselines with no training first, so that I know which score is actually worth something.

Report Structure

Next I explain my EDA and preprocessing, then my tokenizers, then the three models with diagrams, then the comparison of results, and at the end what I learned.

3. Dataset and Preprocessing

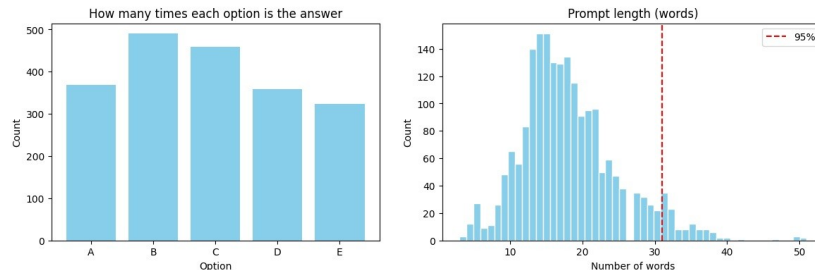
Dataset Description

The training set has 2,000 rows and 8 columns (id, prompt, A, B, C, D, E, answer). The test set has 500 rows and 7 columns. There are no missing values anywhere. There are 242 duplicate prompts in train and 8 in test, which becomes important later. Train has 2,980 unique words and test has 2,975.

Exploratory Data Analysis (EDA)

1. **Answer distribution:** the counts are A = 369, B = 490, C = 459, D = 358, E = 324. B is the most common with 24.5% against a uniform 20%, so the data is a little skewed but not imbalanced enough to need class weights. This gives me my first baseline: always guessing B gives 24.5% accuracy.
2. **Prompt length:** the average is 18.15 words (median 17, max 51) and 95% are under 31 words. This is why I chose max_len = 128 for the word level models.
3. **Option length:** there is a common exam trick that the longest option is usually the right one, so I tested it. Correct options have 28.66 words on average against 25.68 for the wrong ones, and always picking the longest option gives 34.95% accuracy. That is a surprisingly strong baseline with no training at all, so the data does have a length artefact.

4. **Word overlap:** I checked how many words each option shares with the question and picked the highest one. This only gives 9.25%, which is worse than random. I think the reason is that the wrong options are written by rephrasing the question, so they copy the prompt more than the right answer does.
5. **Trick questions:** I searched for NOT, EXCEPT, INCORRECT, FALSE and LEAST, because one negation word flips a whole sentence and that is the classic weakness of a CNN which only sees 2 to 4 words at a time. There are 0 such questions, which removes the main argument against using a CNN here.
6. **Unseen words:** this was the most useful thing I found. The number of test words never seen in training is 0, so the OOV rate is 0.00%. This means the main benefit of subword tokenization, which is splitting unknown words into known pieces, is worth nothing on this dataset. So I could already tell before training that the main advantage of the transformer would not help here.
7. **Train vs test:** the two prompt length distributions sit on top of each other, so both sets come from the same source and validation results should carry over to the leaderboard.



Left: how often each option is the answer. Right: prompt length in words with the 95th percentile marked.

Data Preprocessing

For the from scratch models I lowercase everything (otherwise "The" and "the" take two different slots in the embedding table), I fix smart quotes so that "don't" is one token, I remove URLs, and I collapse extra spaces.

I did not do this cleaning for DeBERTa. Pretrained transformers are trained on normal text with capital letters and their tokenizers treat casing and punctuation as real information, so cleaning would only push the input away from what the model was trained on. I fed it raw text.

Data Augmentation

I used option order shuffling on the two from scratch models. During training the five texts are put in a random order and the label moves with its own option (label = order.index(label)). Since B is the answer 24.5% of the time, a model could otherwise learn that "slot 2 is usually right" and gain marks without understanding anything. Shuffling makes the position useless, so the model has to actually match the question with the option. I only do this on the training split, because randomising the validation set would make the metric noisy.

4. Tokenization Strategy

Primary Tokenizer Selection

For DeBERTa I used the pretrained DeBERTa v3 tokenizer (`AutoTokenizer.from_pretrained("microsoft/deberta-v3-base")`). It loads as `DebertaV2Tokenizer`, a `SentencePiece` subword tokenizer with a 128,000 token vocabulary, about four times bigger than the `WordPiece` of BERT (30k) or the `BPE` of RoBERTa (50k).

For the two from scratch models I wrote my own word level tokenizer. It splits on `\w+`, counts word frequencies, keeps the most common words, and reserves index 0 for `<pad>` and index 1 for `<unk>`. The encode method cuts long texts and pads short ones to exactly `max_len`, which is what lets me stack a batch into one tensor.

Justification

The usual reason to use subword tokenization is to handle unseen words, but as I showed in Section 3 the OOV rate here is 0.00% and the whole vocabulary fits in 2,982 entries. So a word level tokenizer is enough, and it has two advantages: every token is a full word, so a kernel of width 3 really sees a three word phrase, and the sequences are shorter so training is much cheaper. The DeBERTa tokenizer is still the right choice for DeBERTa, because pretrained weights only make sense with the tokenizer they were trained with.

Implementation Details

- **Sentence pairs:** for DeBERTa I pass the prompt and the option as two separate arguments, so it builds a proper `[CLS]` prompt `[SEP]` option `[SEP]` sequence with correct token type IDs. In my own notebooks I had to write `"[SEP]"` as a plain string, which my tokenizer then treats as a normal word. That is a real limitation of doing it by hand.
- **Picking max_len from tokens and not words:** word count is not the same as token count, so instead of reusing 128 I measured the real token length of 2,000 sampled pairs. The mean was 54.1, the 95th percentile 97, the 99th 118 and the max 188. So `max_len = 256` means 100% of examples fit and nothing is ever cut off.
- **Dynamic padding:** padding everything to 256 when the mean is only 54 wastes most of the compute, so I set `padding=False` in the dataset and pad inside a custom `collate_fn` instead. Each batch is padded only to its own longest sequence. A shape check showed a batch of 2 coming out as `[2, 5, 63]`, so 63 tokens instead of 256. That is roughly 4 times less compute for that batch with no information lost.

Experimental Comparison

Property	My SimpleTokenizer	DeBERTa v3 (SentencePiece)
Granularity	Whole words	Subword
Vocabulary cap set / actually used	25,000 / 2,982	128,000 (pretrained)
OOV rate on test	0.00%	0.00%
Sequence length	128, fixed padding	256 cap, dynamic padding (~63 typical)
Separator	literal string, read as a word	real [SEP] token
Best validation MAP@3	1.0000	0.9200

I kept both tokenizers, because each one only works with its own model. The comparison itself is the finding: the big 128k subword vocabulary gave no measurable benefit on a dataset that only has 2,980 words in it.

5. Modeling and Experimentation

All three models share one interface. A question becomes five "prompt [SEP] option" texts, each is encoded on its own, one shared scoring head gives each option a single number, and the five numbers are reshaped into a (batch, 5) logit vector trained with CrossEntropyLoss. Using one shared head is what makes the option shuffling valid. Every model was run with three configurations, logged to Weights and Biases, with seed 42 and an 85/15 stratified split (1,700 train / 300 validation).

5.1 Model 1: Fine Tuned Transformer (DeBERTa v3 Base)

- **Architecture:** I fine tuned Microsoft DeBERTa v3 Base [1] using AutoModelForMultipleChoice [7] from Hugging Face, which adds a pooler and a Linear(768 to 1) head and does the reshape to (batch, 5) itself. The model has 12 layers, hidden size 768 and 12 attention heads. The load report confirmed it worked correctly: the pretraining heads were dropped as UNEXPECTED, and the classifier and pooler were listed as MISSING and freshly initialised, which is what should happen for a new task.
- **Salient points:** DeBERTa uses disentangled attention, which means every token has a separate content vector and position vector, and attention is the sum of content to content, content to position and position to content terms. This helps here because what matters is the relative distance between a phrase in the question and a phrase in the option across the [SEP], and not their absolute positions. More importantly, since the prompt and the option go in as one joint sequence, every option token can attend to every question token from the first layer. My CNN and my LSTM cannot do this kind of cross matching at all.
- **Fine tuning strategy:** I used AdamW [5] with a linear schedule and 10% warmup for 3 epochs. Weight decay of 0.01 was applied using two parameter groups (76 with decay, 126 without), leaving out bias and LayerNorm weights because those should not be shrunk. I used batch size 8 with gradient accumulation over 2 steps for an effective batch of 16, fp16 mixed precision, and gradient clipping at 1.0 after scaler.unscale_(). I also tried freezing the embedding layer, and unlike the usual result this made things much better.
- **Ensembling:** I averaged the probabilities of all three runs, but I checked the ensemble on validation first. It got 0.9017 against 0.9200 for the single best model, so my code correctly set use_ensemble = False. Averaging one good model with two bad ones just drags the good one down, so ensembling is not automatically helpful.

5.2 Model 2: Custom Deep Learning Model (TextCNN)

- **Architecture:** I built a TextCNN [3] for option scoring. The five options are folded into the batch as (batch x 5, seq_len) so that all five go through the same weights in one pass. Embeddings are 128 dimensional with padding_idx = 0, then three parallel convolutions with kernel widths 2, 3 and 4, each with 100 filters and a ReLU. Each branch is max pooled over time, the three 100 dim outputs are concatenated into 300 dims, and then there is dropout and the shared Linear(300 to 1) scorer.
- **Salient points:** a kernel of width k makes each filter a k-gram detector, so running 2, 3 and 4 together lets the model spot short technical terms and longer phrases at the same time instead of committing to one length. The important detail is masked max pooling. A normal max pool over a padded sequence can pick a padding position, which lets the padding embedding affect the score. So I build a mask, pad it to match the convolution output length, and fill the masked positions with -1e4 before pooling. I used -1e4 and not -inf on purpose, because -inf gives NaNs if a whole window is masked. I chose max pooling over mean pooling because the evidence for an answer is local: one matching phrase should be enough, and averaging would water it down.
- **Experiments:** (1) baseline, embed 128, 100 filters, kernels [2,3,4], dropout 0.3, lr 1e-3; (2) bigger and more regularised, embed 200, 150 filters, dropout 0.4, lr 5e-4; (3) kernels changed to [3,4,5] to test whether longer phrases help, which is the experiment that is specific to a CNN. All of them used AdamW, batch size 32 and 15 epochs.

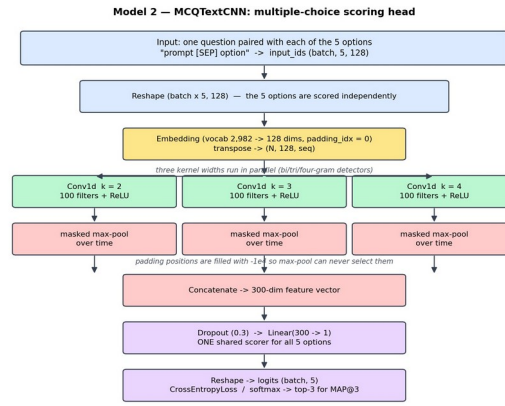


Figure 5.2: MCQTextCNN architecture.

5.3 Model 3: RNN Based Model (Bi-LSTM from Scratch)

- Architecture:** my sequence model was a bidirectional LSTM [4] written by hand instead of using nn.LSTM. For each direction I keep raw nn.Parameter tensors: W_x of shape (embed_dim, 4 x hidden), W_h of shape (hidden, 4 x hidden), and a bias of length 4 x hidden. One timestep computes $gates = x_t @ W_x + h_{t-1} @ W_h + b$, splits it into the input, forget, candidate and output gates, and then applies $c_t = f * c_{t-1} + i * g$ and $h_t = o * \tanh(c_t)$. The two directions have completely separate weights, their outputs are mask aware pooled and concatenated into 256 dims, and the shared Linear(256 to 1) scorer gives the five logits.
- Salient points:** a sequence model makes sense here because correctness depends on word order and not only on which words appear. "A causes B" and "B causes A" have the same words but only one is right, and the wrong options in this dataset are often exactly that kind of reordering. I used a bidirectional model because a left to right LSTM does not know how the sentence ends when it is reading the start, and the clause that decides correctness is often at the end of an option. Three implementation choices mattered a lot. The four gates are fused into one matrix, so each timestep is one big matmul instead of four small ones, which is faster on a GPU. The forget gate bias starts at 1.0 [6], because at $\text{sigmoid}(0) = 0.5$ the cell state halves at every step and memory dies out before training can fix it. And gradient clipping at 1.0 is essential, because the repeated multiplication through the recurrence is exactly where gradients explode. Weights are initialised uniformly in plus or minus $1/\sqrt{\text{hidden}}$, which is what the PyTorch LSTM does itself, so the comparison is fair.
- Experiments:** (1) baseline, embed 128, hidden 128, bidirectional, dropout 0.3, lr $1e-3$; (2) bigger, embed 200, hidden 192, dropout 0.4, lr $5e-4$; (3) smaller and unidirectional, embed 100, hidden 96, dropout 0.2, lr $2e-3$, which is the ablation that tests whether the backward pass is actually doing anything.

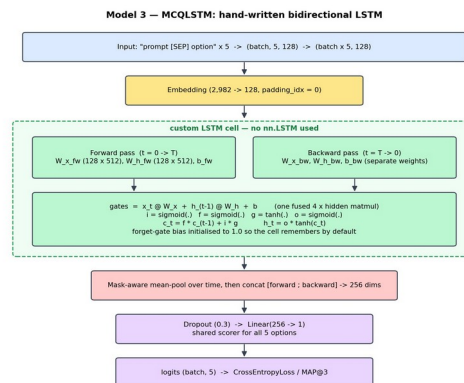


Figure 5.3: MCQLSTM, my hand written bidirectional LSTM.

6. Performance and Comparative Analysis

Evaluation Metrics

I evaluated everything with MAP@3 as the competition requires, and I also logged accuracy and macro F1, because macro F1 would show me if a model collapsed onto one popular letter. My baselines before training were: random MAP@3 = 0.3667, random accuracy 20%, always guessing B 24.5%, longest option 34.95%, and word overlap 9.25%.

Training Performance

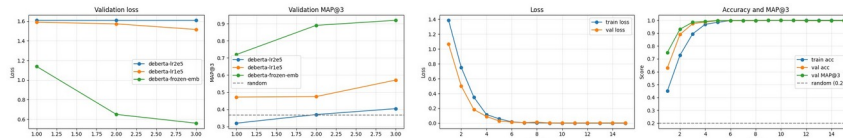
I logged all nine runs to Weights and Biases [9]. The from scratch models converged quickly and cleanly, while the transformer was actually unstable, which is the opposite of what I expected.

DeBERTa run	LR	Frozen emb.	Ep 1	Ep 2	Ep 3	Best MAP@3	Best acc.
deberta-lr2e5	2e-5	No	0.3189	0.3694	0.4039	0.4039	0.2300
deberta-lr1e5	1e-5	No	0.4717	0.4744	0.5711	0.5711	0.3967
deberta-frozen-emb	2e-5	Yes	0.7200	0.8906	0.9200	0.9200	0.8600

The first run completely collapsed. Its training loss sat at 1.6113, 1.6118, 1.6124, basically constant at $\ln(5) = 1.6094$, which is the loss of a uniform guess over five options. It learned nothing in three epochs and 0.4039 is barely above the 0.3667 random baseline. I think the cause is instability from a fairly high learning rate together with fp16 on a small dataset. Halving the rate to 1e-5 got it learning again, but three epochs were still not enough.

Freezing the embedding layer changed everything, taking the score from 0.4039 to 0.9200 at the same learning rate. The reason is that the embedding matrix of DeBERTa is 128,000 x 768, which is about 98M parameters and by far the biggest block in the model. Updating that with only 1,700 examples is mostly noise. Freezing it removes about half of the trainable parameters and lets the optimiser spend its budget on the transformer layers and the new head instead.

Both from scratch models reached 1.0000 validation MAP@3 in all three configurations. For the TextCNN the baseline was the fastest, reaching 0.9789 after a single epoch and 1.0000 by epoch 4, and changing the kernels from [2,3,4] to [3,4,5] made no difference at all, which suggests the useful signal is in short 2 to 3 word spans. The Bi-LSTM was slower, climbing 0.7500, 0.9306, 0.9850, 0.9911, 0.9983 over five epochs and hitting 1.0000 at epoch 9. That is expected, because recurrence is sequential so the gradient path back to an early token is much longer than through one convolution. The unidirectional version also reached 1.0000, so the backward pass was not actually needed here.



WandB curves. Left: validation loss and MAP@3 for the three DeBERTa runs. Right: loss and accuracy/MAP@3 for my best Bi-LSTM run, with the 0.20 random accuracy line.

Comparative Report

Model Architecture	Val MAP@3	Key Observation
Bi-LSTM (from scratch)	1.0000	Perfect in all 3 configs, slowest to converge (epoch 9), unidirectional version also perfect.
TextCNN	1.0000	Perfect in all 3 configs, fastest overall, kernel width made no difference.
DeBERTa v3 Base (frozen emb.)	0.9200	Best transformer run, freezing embeddings was the key change.
DeBERTa v3 Base (lr 1e-5)	0.5711	Learning but nowhere near converged in 3 epochs.
DeBERTa v3 Base (lr 2e-5)	0.4039	Collapsed, loss stuck at $\ln(5)$, barely above random.
Ensemble of 3 DeBERTa runs	0.9017	Worse than the single best model, so I rejected it.
Longest option heuristic	34.95% acc.	Strongest baseline with no training at all.

Why did the small models win? I think there are four reasons. First, the dataset is small and closed: 2,000 rows, 2,980 words and 0% OOV, so the broad language knowledge of the transformer has nothing to do here. Second, the data is templated and has 242 duplicate prompts, which can end up on both sides of a random split, so a model that can memorise 1,700 short examples can hit 1.0 without really reasoning. Third, DeBERTa has around 184M parameters for 1,700 examples, which is roughly 100,000 parameters per row, against about 0.2M for my CNN and LSTM. That is also why freezing 98M embedding parameters helped so much. Fourth, the budgets were not equal: 15 epochs for the small models against only 3 for the transformer, and the lr 1e-5 run was still improving when it stopped, so part of the gap is my fault and not the fault of the architecture.

So I do not think the right conclusion is "CNNs beat transformers". A validation score of 1.0000 is really a warning that my validation set has stopped telling me anything, because I cannot rank six tied runs, and the leaderboard score of 0.7581 confirms that it was inflated by leakage. The honest finding is that **on a small, closed vocabulary, templated dataset a 0.2M parameter model matches a 184M parameter one, and capacity should match the data instead of being maximised**. The cost difference makes this even clearer: my small models train in seconds per epoch, while DeBERTa needed batch size 8 with gradient accumulation and fp16 just to fit in memory and took minutes per epoch, for a lower score.

As a last check I plotted how often each letter was my top-1 prediction on the test set: A = 86, B = 114, C = 120, D = 96, E = 84. This follows the training distribution and shows the model did not collapse onto one option.

Kaggle Performance

I made the submission by ranking the top three options with `np.argsort(-probs)` and writing them as space separated letters (submission.csv, 500 x 2). Two details matter for a valid submission and I handled both: `shuffle=False` in the test DataLoader (predictions are matched to IDs by position) and `index=False` when writing the CSV.

My final submission scored 0.75810 MAP@3 on the Kaggle leaderboard and finished at rank 164.

The gap between this and my validation score is the most informative number in the whole project. My models scored 1.0000 on validation but only 0.7581 on the leaderboard, a drop of about 0.24. This is direct evidence for what I suspected in Section 3: the 242 duplicate prompts leak across a random split, so my validation set was partly made of questions the model had already memorised. The leaderboard has no such duplicates, so it shows the real performance. 0.7581 is still well above the 0.3667 random baseline, so the models did learn something real, but nothing close to what validation suggested.

7. Conclusion and Future Work

Key Learnings

The biggest thing I learned is that model capacity has to match the data and not be maximised. Freezing the embedding layer of DeBERTa took validation MAP@3 from 0.4039 to 0.9200, so removing 98M trainable parameters made the model much better. I also learned that the problem formulation mattered more than the architecture. Once the option scoring interface was set up, swapping a hand written LSTM for a 184M parameter transformer was a small change, and that is what made the comparison fair. Writing the LSTM by hand taught me things the library hides, like why the forget gate bias starts at 1.0 and why gradient clipping is not optional in a recurrent model, and getting the same score as the library models told me my implementation was correct. Finally, a perfect score is a signal to investigate and not a result to celebrate. My validation said 1.0000 and the leaderboard said 0.7581, and tracing that gap back to the EDA (0% OOV, 2,980 words, 242 duplicate prompts) taught me more than any tuning I did.

Challenges Faced

The hardest problem was the training collapse in my first DeBERTa run, where the loss was stuck at $\ln(5)$ for all three epochs. To diagnose it I had to recognise 1.6094 as the uniform guess loss instead of thinking it was just a slow start. I fixed it by lowering the learning rate and then properly by freezing the embeddings. I also made sure `scaler.unscale_()` is called before clipping, because clipping scaled gradients would use the wrong threshold without any error.

I also ran into GPU memory limits, because with five options a batch of 8 questions is really 40 sequences through the encoder. I used gradient accumulation over 2 steps, fp16, and explicit cleanup between runs.

A smaller but annoying bug was padding leaking into predictions in the CNN, where an unmasked max pool can pick a padded position. Fixing it meant masking the convolution output, and the mask has to be padded first because padding = k-1 makes the output longer than the input.

Areas for Improvement

- **A validation set that actually separates models.** This is the main thing, and the 0.24 gap between validation and the leaderboard proves it. With six runs tied at 1.0000 I cannot rank my own models. I would use 5 fold cross validation grouped by prompt (GroupKFold) so that the 242 duplicate prompts cannot land on both sides of a split, and report the mean and the standard deviation across folds. This should give a validation score close to the 0.7581 I actually got.
- **A fair budget for the transformer.** I would retrain DeBERTa for 8 to 10 epochs with early stopping on validation MAP@3, use layer wise learning rate decay, and sweep the learning rate properly instead of trying only two values. Since the leaderboard shows that the real task is harder than validation suggested, the cross attention of DeBERTa may actually be worth its cost after all.
- **A leakage ablation.** Remove the 242 duplicate prompts and retrain everything. If validation drops towards 0.7581, that confirms memorisation was doing the work and gives me a trustworthy validation set to tune against.
- **A harder test set.** Rephrasing held out questions, or putting in wrong options taken from other questions, would show whether the models learned meaning or just word overlap. Given the 0% OOV rate I expect the word level models to fall apart much more than DeBERTa.
- **A better from scratch tokenizer.** Adding [SEP] as a real reserved token plus a segment embedding that marks question against option would let the model tell the two halves apart properly.
- **Weighted ensembling.** My equal weight average was pulled down by two weak runs. Tuning the blend weights on validation with something like Optuna would let the strong model dominate and would probably have beaten 0.9200.

8. References

- [1] He, P., Gao, J., and Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543. Model card: microsoft/deberta-v3-base, Hugging Face Model Hub.
- [2] He, P., Liu, X., Gao, J., and Chen, W. (2020). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. arXiv:2006.03654.
- [3] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP 2014. arXiv:1408.5882.
- [4] Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735 to 1780.
- [5] Loshchilov, I., and Hutter, F. (2017). Decoupled Weight Decay Regularization (AdamW). arXiv:1711.05101.
- [6] Gers, F. A., Schmidhuber, J., and Cummins, F. (2000). Learning to Forget: Continual Prediction with LSTM. Neural Computation, 12(10), 2451 to 2471.
- [7] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020 (System Demonstrations).
- [8] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.
- [9] Biewald, L. (2020). Experiment Tracking with Weights and Biases. wandb.ai.

9. Important Links

- **Weights and Biases project (all 9 runs):** wandb.ai/22f1000828-t1-2026-indian-institute-of-technology-madras/DL-22f1000828-notebook-t22026
- **Kaggle notebook, Model 1 (DeBERTa v3 fine tuning):** kaggle.com/code/ashritakhanta/dl-22f1000828-notebook-t22026?scriptVersionId=337886436
- **Kaggle notebook, Model 2 (custom TextCNN):** kaggle.com/code/ashritakhanta/dl-22f1000828-notebook-t22026?scriptVersionId=337998076
- **Kaggle notebook, Model 3 (Bi-LSTM from scratch):** kaggle.com/code/ashritakhanta/dl-22f1000828-notebook-t22026?scriptVersionId=337870064
- **GitHub repository (project code):** github.com/22f1000828/DL-22f1000828-notebook-t22026